Practice Assignment

Transactions · Joins · Indexing · Query Optimization

PART 1

```
CREATE database FoodDeliviery;
USE FoodDeliviery;
CREATE table customers(
id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
email VARCHAR(100) UNIQUE,
signup_date DATE
);
CREATE TABLE restaurants (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   cuisine VARCHAR(50)
);
CREATE TABLE orders (
   id INT PRIMARY KEY AUTO_INCREMENT,
   customer_id INT,
   restaurant_id INT,
   total DECIMAL(10,2),
   status VARCHAR(30),
   created_at DATETIME,

   FOREIGN KEY(customer_id) REFERENCES customers(id),
   FOREIGN KEY(restaurant_id) REFERENCES restaurants(id)
);
CREATE TABLE accounts (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(50),
   balance DECIMAL(10,2)
);

INSERT INTO customers(name, email, signup_date) VALUES
('Aarav Patel','aarav@gmail.com','2025-01-05'),
('Riya Shah','riya@gmail.com','2025-01-08'),
('Vivaan Mehta','vivaan@gmail.com','2025-01-12'),
('Ananya Joshi','ananya@gmail.com','2025-01-18'),
('Dhruv Patel','dhruv@gmail.com','2025-01-25'),
('Priya Desai','priya@gmail.com','2025-02-01'),
('Yash Trivedi','yash@gmail.com','2025-02-06'),
('Neha Sharma','neha@gmail.com','2025-02-12'),
('Harsh Patel','harsh@gmail.com','2025-02-20'),
('Krishna Shah','krishna@gmail.com','2025-02-25'),
```

('Sneha Patel','sneha@gmail.com','2025-03-01'),
('Rahul Verma','rahul@gmail.com','2025-03-06'),
('Pooja Singh','pooja@gmail.com','2025-03-10'),
('Karan Joshi','karan@gmail.com','2025-03-15'),
('Mihir Shah','mihir@gmail.com','2025-03-20'),
('Nidhi Patel','nidhi@gmail.com','2025-03-25'),
('Jay Mehta','jay@gmail.com','2025-04-02'),
('Ishita Shah','ishita@gmail.com','2025-04-07'),
('Rohan Patel','rohan@gmail.com','2025-04-15'),
('Aditi Desai','aditi@gmail.com','2025-04-20');
INSERT INTO restaurants (name,cuisine) VALUES
('Pizza Hub','Italian'),
('Burger Point','Fast Food'),
('Spice Villa','Indian'),
('Dragon Wok','Chinese'),
('Taco Fiesta','Mexican'),
('Sushi World','Japanese'),
('Punjabi Tadka','Punjabi'),
('Cafe Aroma','Cafe'),
('BBQ Nation','Barbecue'),
('South Treat','South Indian');

INSERT INTO accounts(name,balance) VALUES
('Alice',5000),
('Bob',3000);

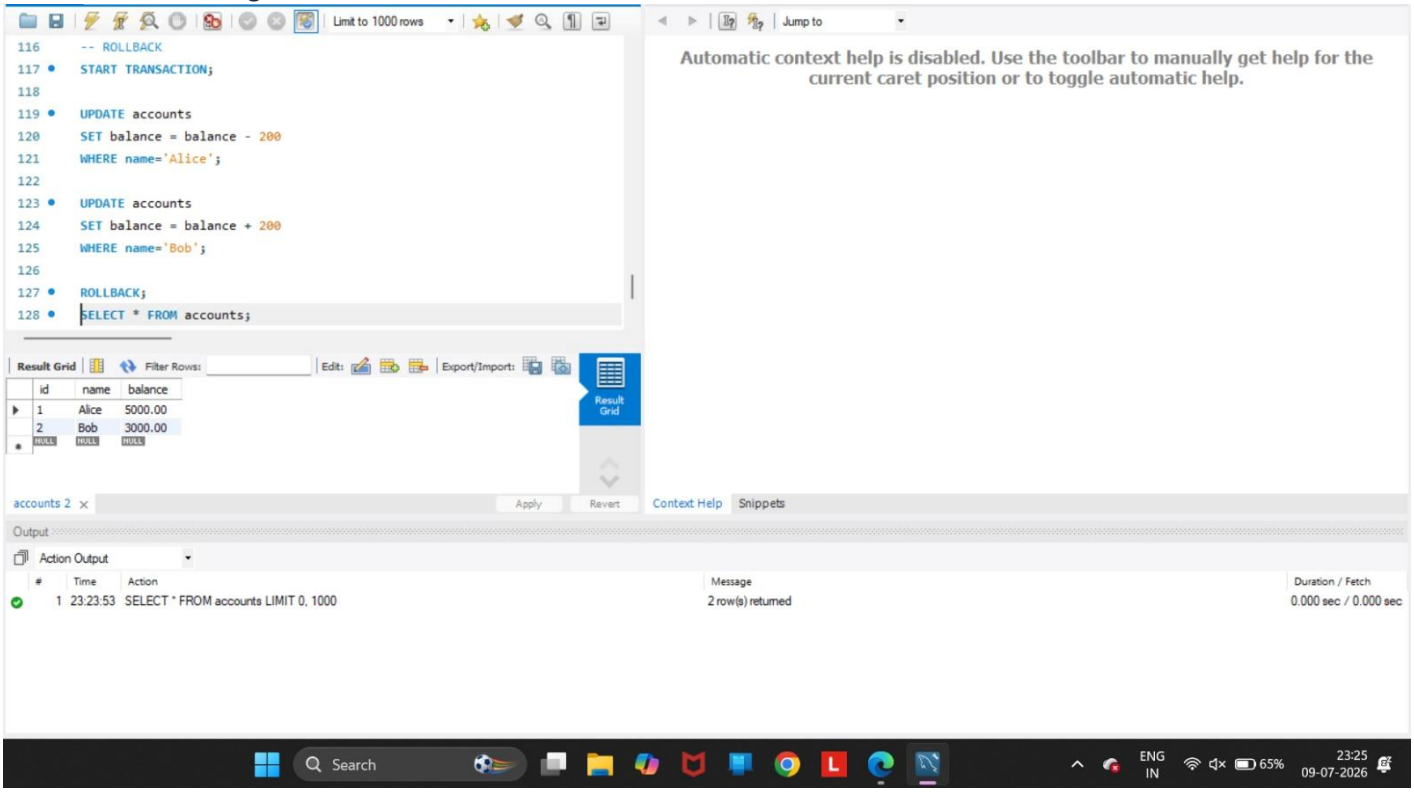
INSERT INTO orders (customer_id,restaurant_id,total,status,created_at) VALUES
(1,1,450.50,'Completed','2025-05-01 12:00:00'),
(1,3,320.00,'Completed','2025-05-05 14:00:00'),
(2,2,280.00,'Pending','2025-05-02 13:00:00'),
(2,5,600.00,'Completed','2025-05-08 18:00:00'),
(3,4,390.00,'Cancelled','2025-05-03 16:00:00'),
(3,1,250.00,'Completed','2025-05-10 19:00:00'),
(4,7,550.00,'Completed','2025-05-04 20:00:00'),
(4,3,450.00,'Pending','2025-05-09 11:30:00'),
(5,8,700.00,'Completed','2025-05-06 17:00:00'),
(5,2,260.00,'Completed','2025-05-12 18:20:00'),
(6,6,800.00,'Completed','2025-05-07 13:30:00'),
(6,9,950.00,'Pending','2025-05-15 20:30:00'),
(7,10,350.00,'Completed','2025-05-08 09:20:00'),
(7,5,420.00,'Completed','2025-05-18 12:15:00'),
(8,4,500.00,'Cancelled','2025-05-09 15:10:00'),
(8,1,620.00,'Completed','2025-05-20 18:45:00'),
(9,3,710.00,'Pending','2025-05-11 11:00:00'),
(9,8,380.00,'Completed','2025-05-22 17:30:00'),
(10,2,450.00,'Completed','2025-05-12 19:20:00'),
(10,6,900.00,'Completed','2025-05-24 21:00:00'),
(11,7,520.00,'Pending','2025-05-13 10:00:00'),
(11,4,610.00,'Completed','2025-05-26 13:00:00'),
(12,5,450.00,'Completed','2025-05-14 14:00:00'),
(12,8,780.00,'Completed','2025-05-28 18:00:00'),
(13,9,840.00,'Pending','2025-05-15 16:00:00'),

(13,1,370.00,'Completed','2025-05-29 20:00:00'),
(14,2,490.00,'Completed','2025-05-16 12:00:00'),
(14,6,860.00,'Cancelled','2025-05-30 19:00:00'),
(15,3,540.00,'Completed','2025-05-17 15:00:00'),
(15,10,310.00,'Completed','2025-06-01 18:00:00'),
(16,7,650.00,'Pending','2025-05-18 17:00:00'),
(16,5,290.00,'Completed','2025-06-02 20:00:00'),
(17,8,720.00,'Completed','2025-05-19 11:00:00'),
(17,9,900.00,'Completed','2025-06-03 13:00:00'),
(18,1,470.00,'Completed','2025-05-20 14:30:00'),
(18,2,350.00,'Pending','2025-06-04 19:30:00'),
(19,4,510.00,'Completed','2025-05-21 10:15:00'),
(19,6,680.00,'Completed','2025-06-05 16:30:00'),
(20,3,450.00,'Completed','2025-05-22 12:30:00'),
(20,7,760.00,'Cancelled','2025-06-06 18:30:00'),
(1,5,550.00,'Completed','2025-06-07 12:00:00'),
(2,8,620.00,'Completed','2025-06-08 14:00:00'),
(3,9,710.00,'Pending','2025-06-09 16:00:00'),
(4,10,450.00,'Completed','2025-06-10 18:00:00'),
(5,6,830.00,'Completed','2025-06-11 20:00:00'),
(6,2,310.00,'Completed','2025-06-12 13:00:00'),
(7,4,480.00,'Completed','2025-06-13 15:00:00'),
(8,7,690.00,'Pending','2025-06-14 17:00:00'),
(9,1,520.00,'Completed','2025-06-15 19:00:00'),
(10,5,410.00,'Completed','2025-06-16 21:00:00');

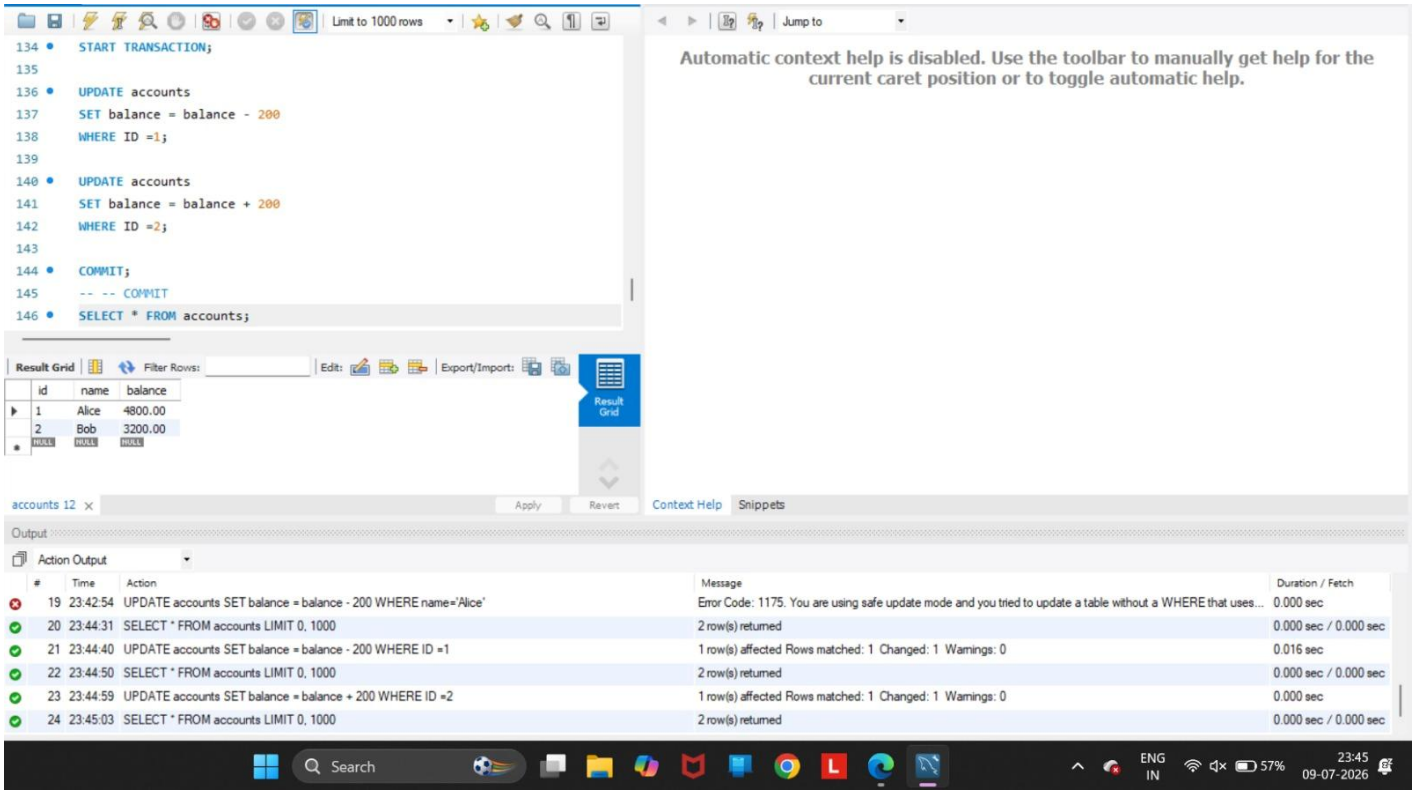
PART 2

TASK 1 Transactions (ACID)

a) Write a transaction that transfers ₹200 from one account to the other, but ends in ROLLBACK. Confirm both balances are unchanged afterward.



b) Write the same transfer again, this time ending in COMMIT. Confirm both balances have changed.

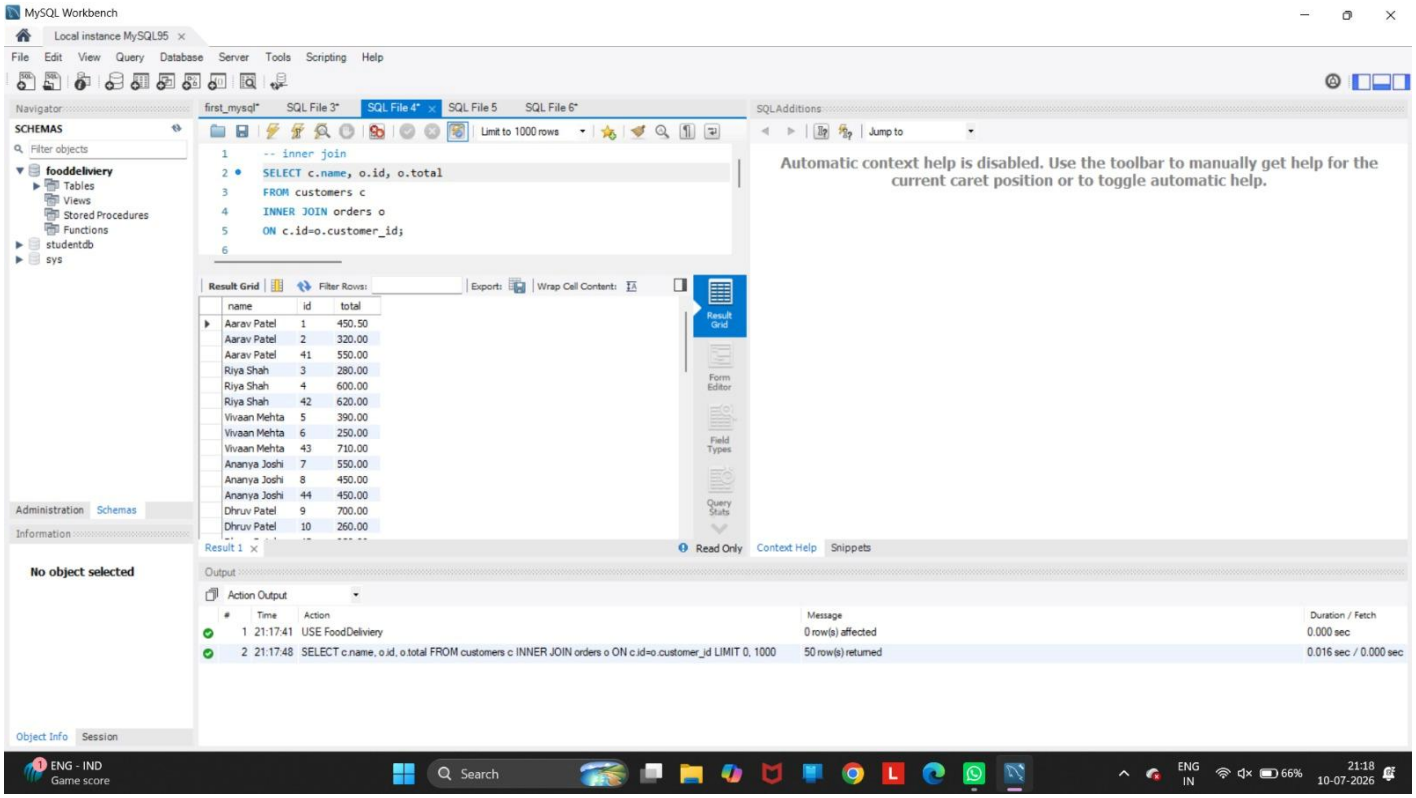


c) In 2–3 sentences, explain which ACID property guarantees the money is never "lost" in the middle of a transfer.

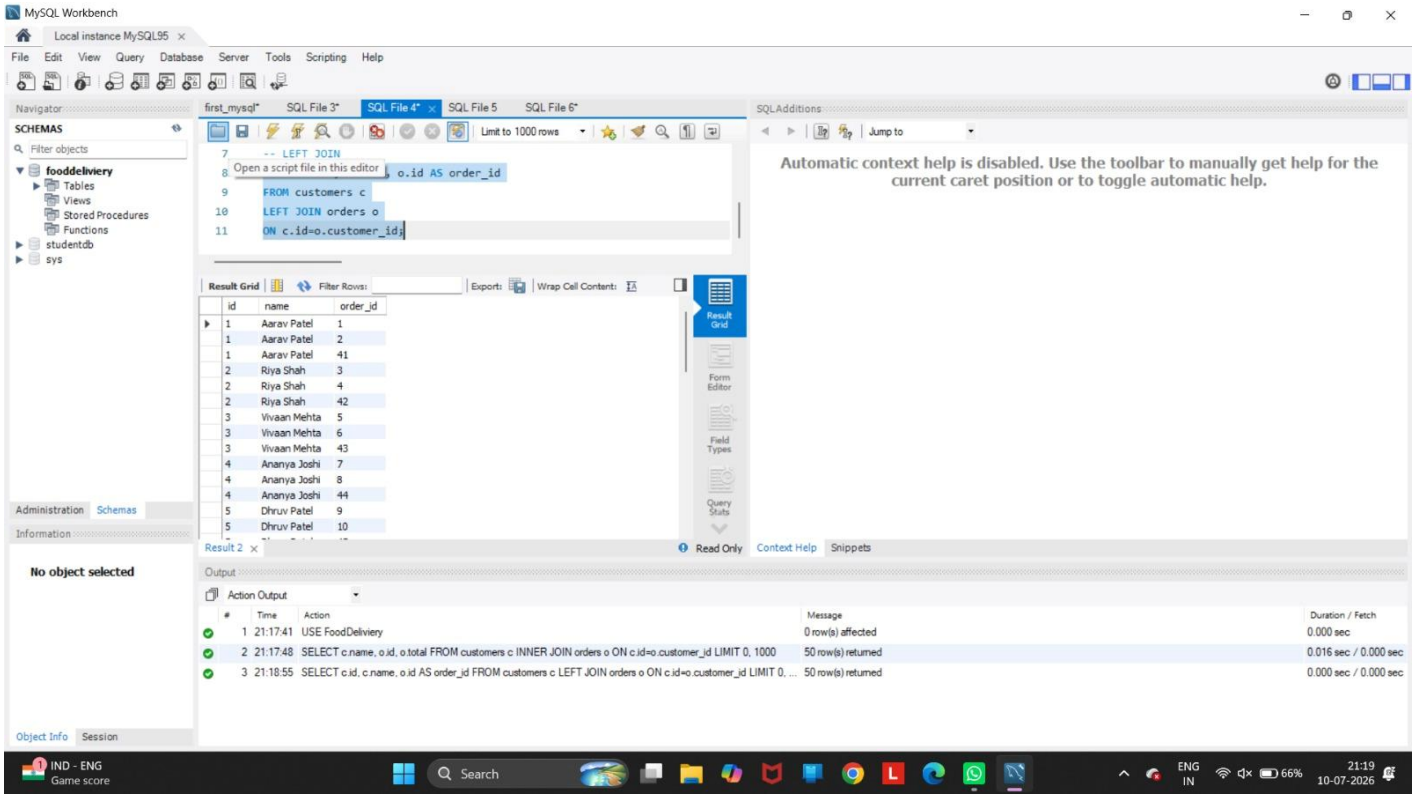
Atomicity is the ACID property that ensures a transaction is completed entirely or not at all. During a money transfer, both the debit and credit operations are treated as a single unit. If any part of the transaction fails, the entire transaction is rolled back, ensuring that no money is lost or partially transferred.

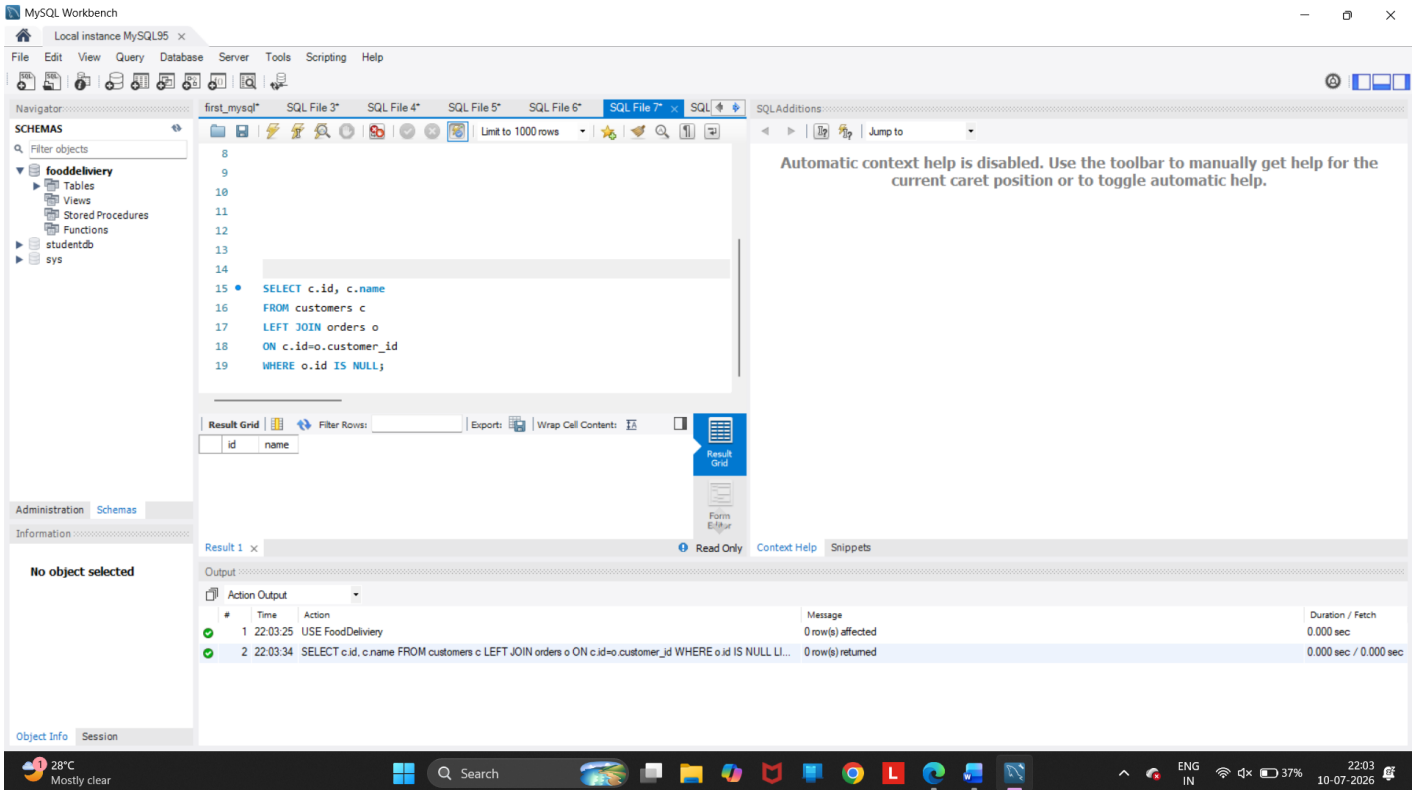
TASK 2 Joins

a) Write an INNER JOIN that lists every customer who has placed at least one order.



b) Write a LEFT JOIN that lists every customer, including those with zero orders. Then add a WHERE clause to isolate only the customers who have never ordered.





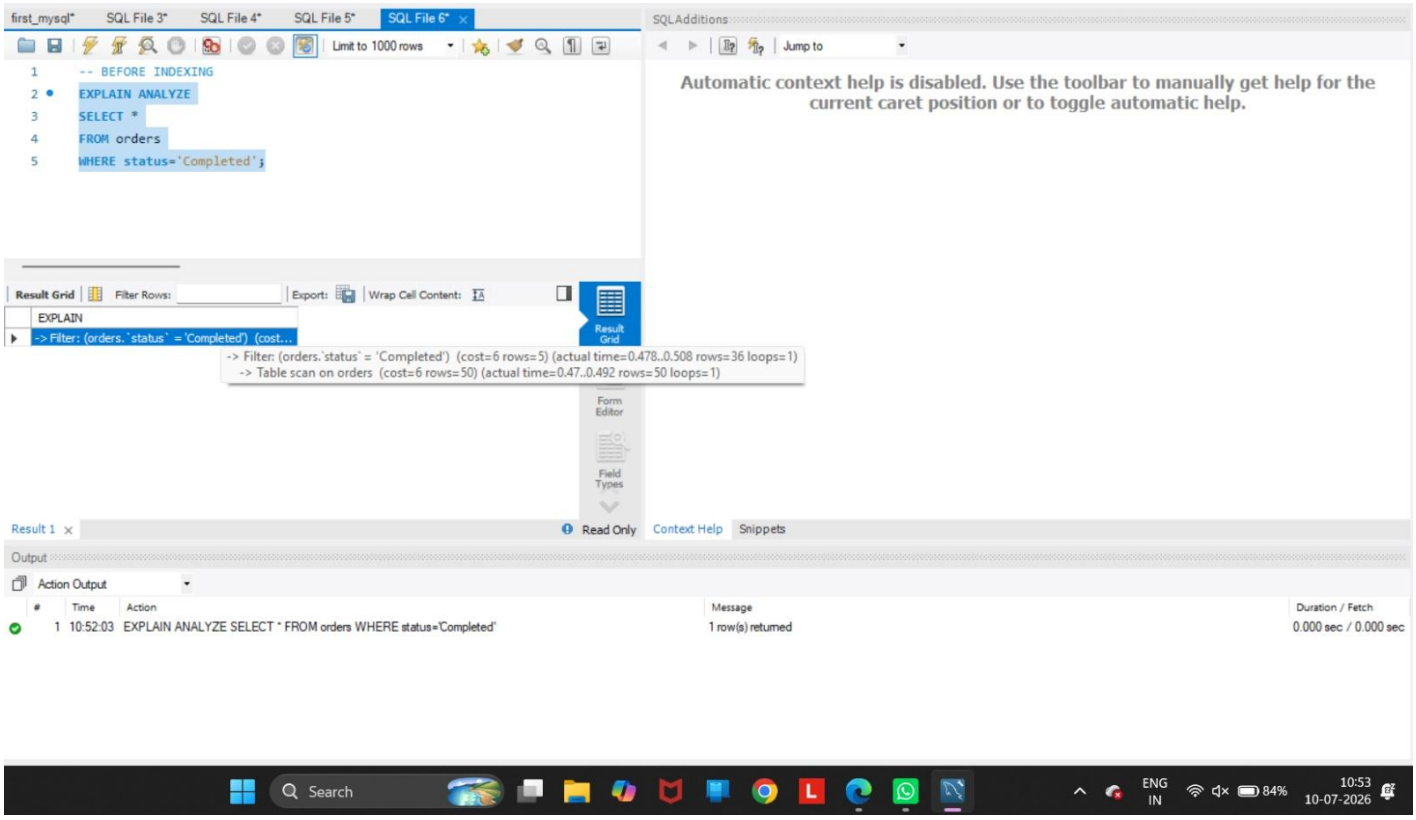
c) In one sentence, explain why a marketing team would prefer the result of query (b) over query (a).

A marketing team would prefer the LEFT JOIN result because it includes customers who have never placed an order, allowing the company to target them with promotional offers, discounts, or reminder campaigns.

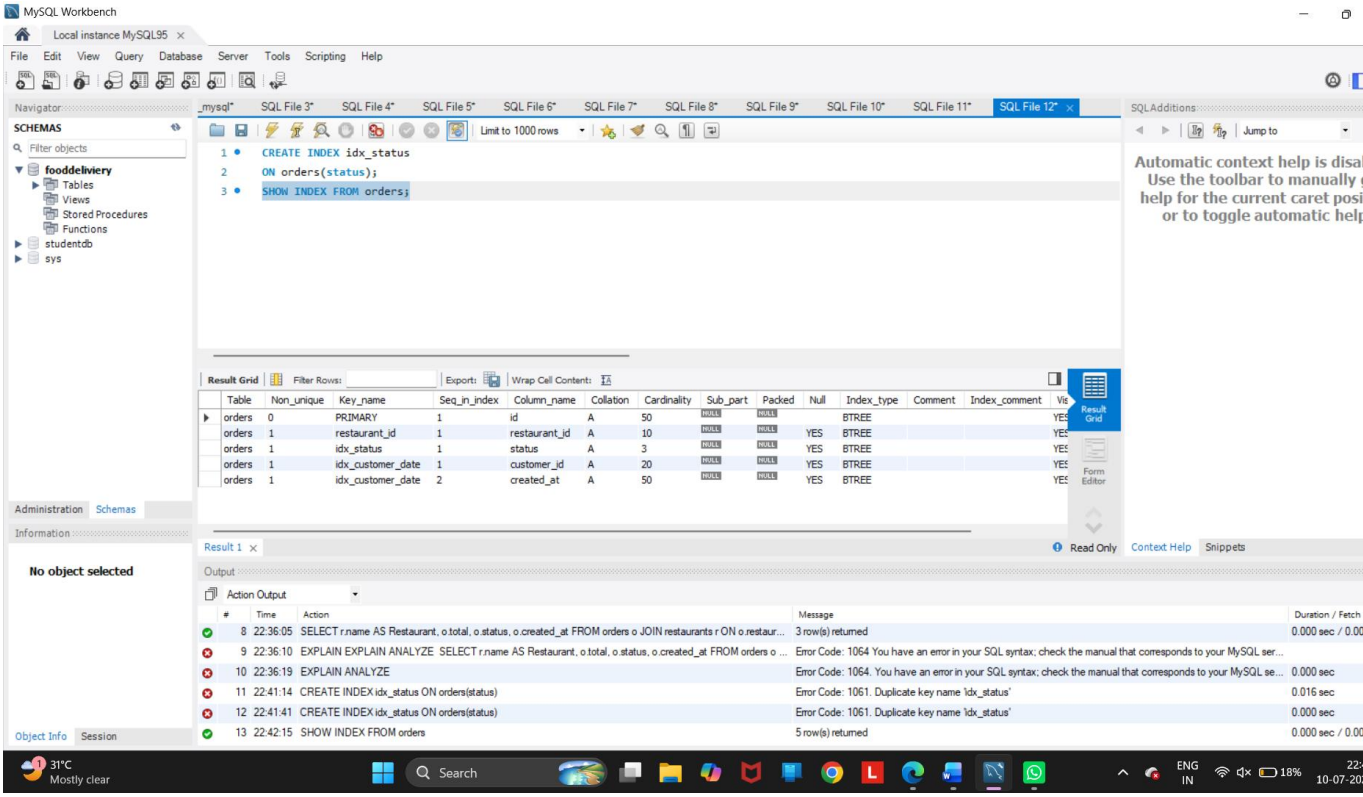
TASK 3 Indexing

a) Pick a column that is not currently indexed (for example status or created_at).

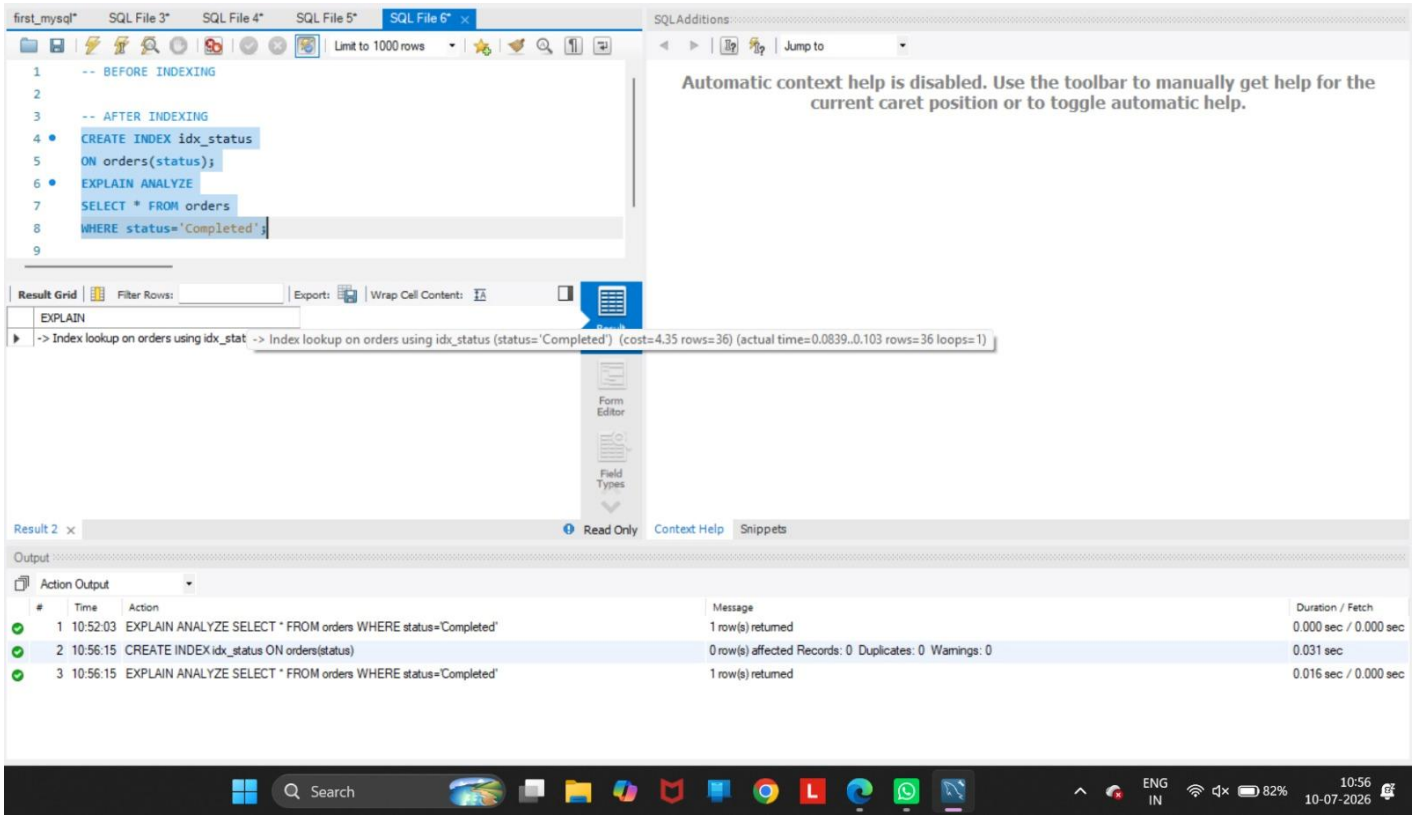
b) Run EXPLAIN ANALYZE on a query that filters by that column, and record the rows examined and actual time.



c) Create an index on that column.



d) Re-run the exact same query and record the new numbers.

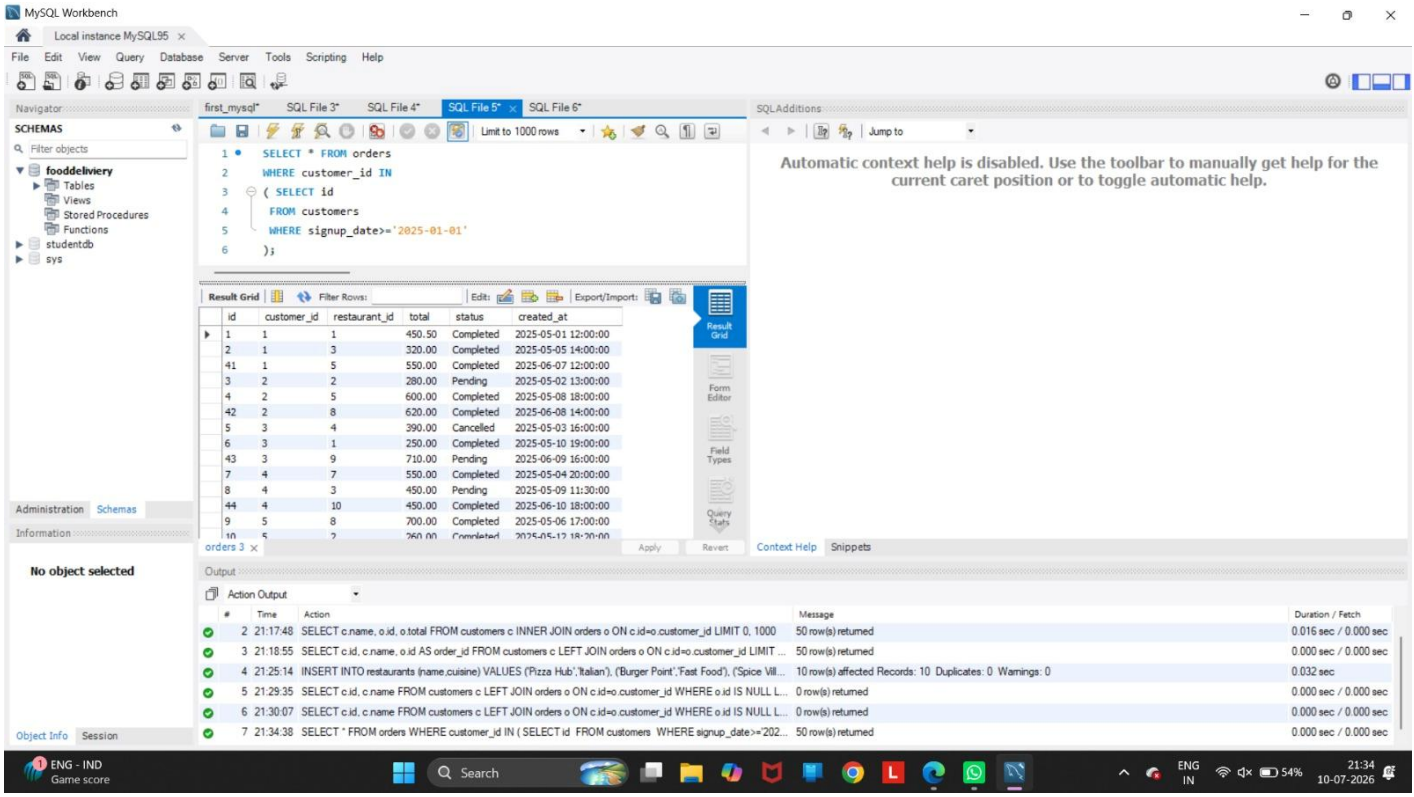


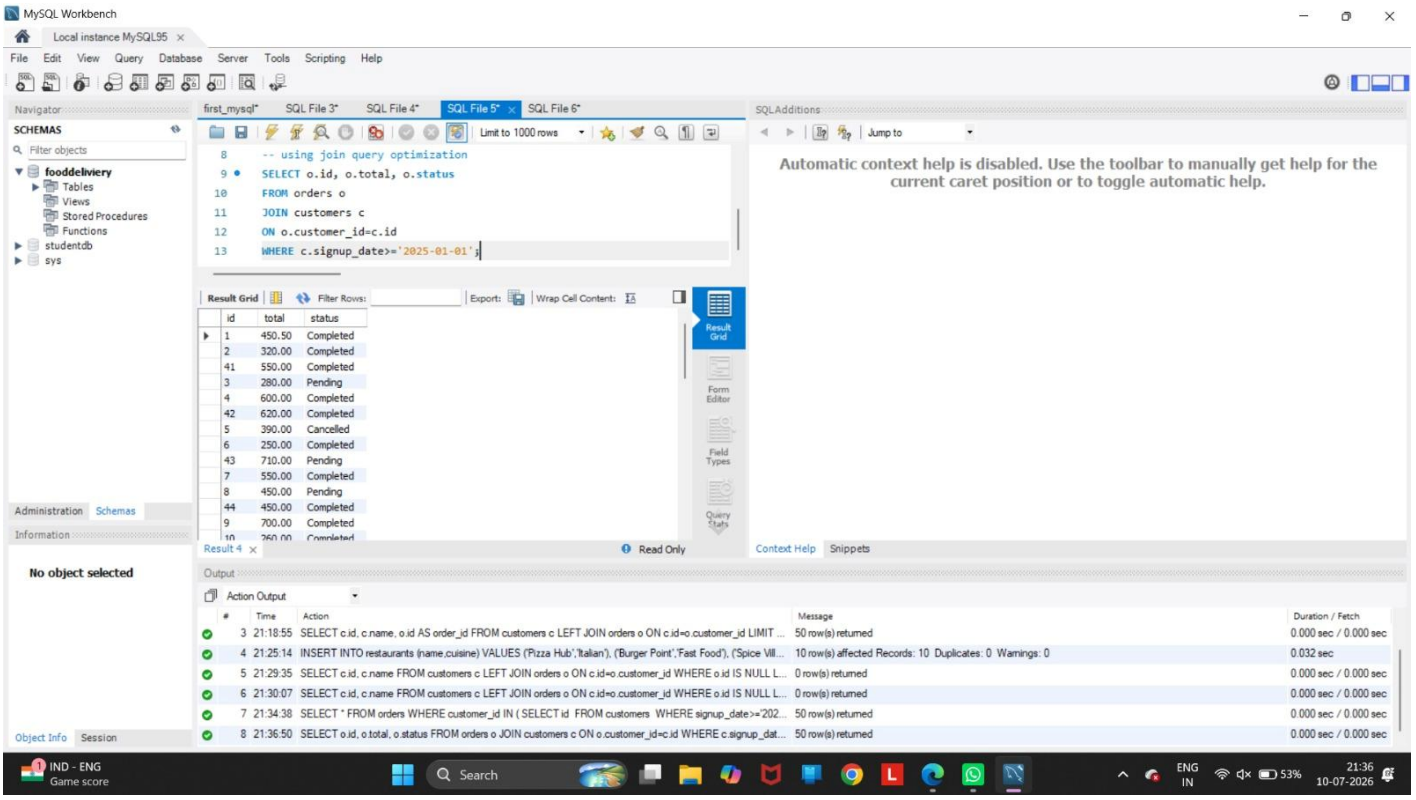
e) Submit a one-line comparison of before vs. after.

After creating the index on the `status` column, MySQL used the `idx_status` index to locate matching rows more efficiently, reducing the amount of work required and improving query performance.

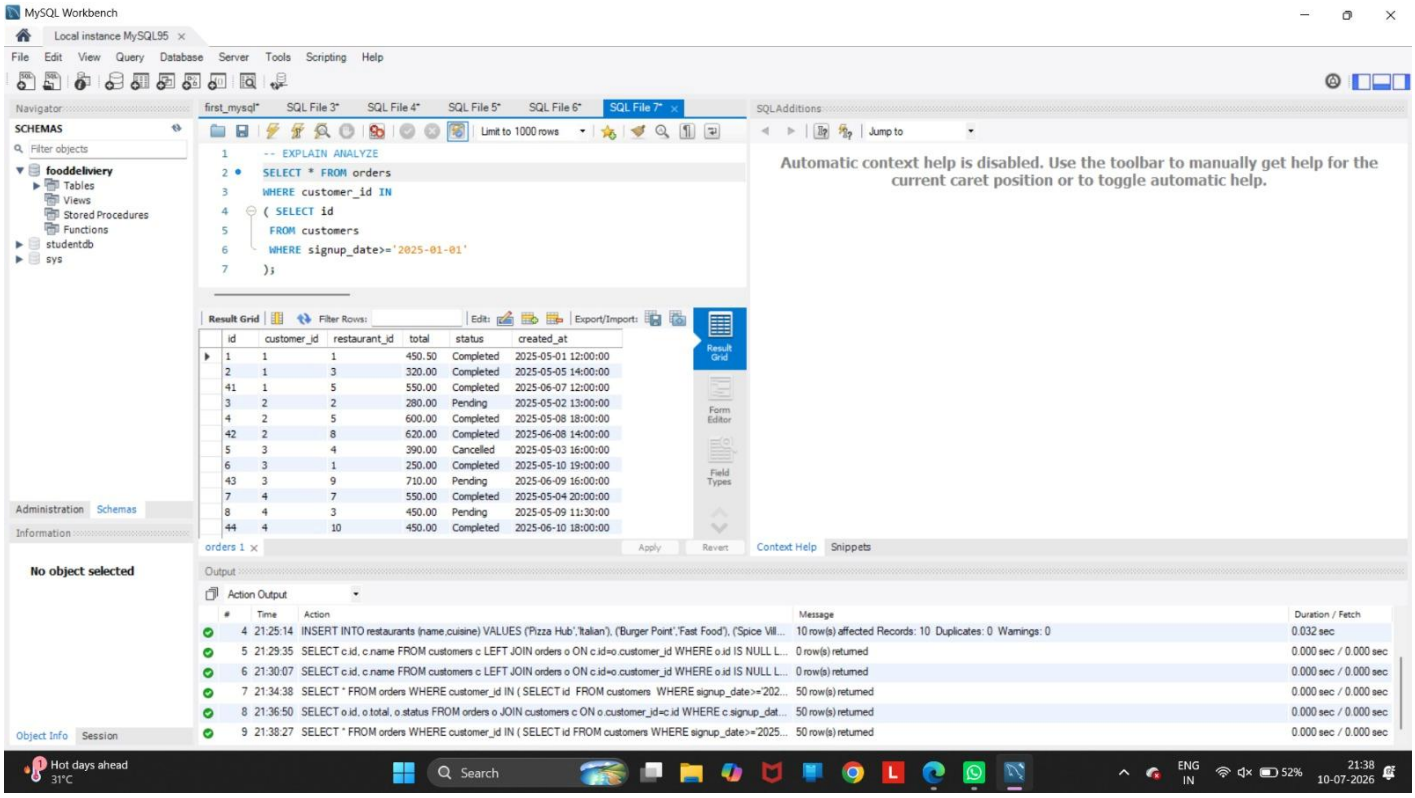
TASK 4 Query Optimization

a) Rewrite it as a JOIN instead of a subquery.

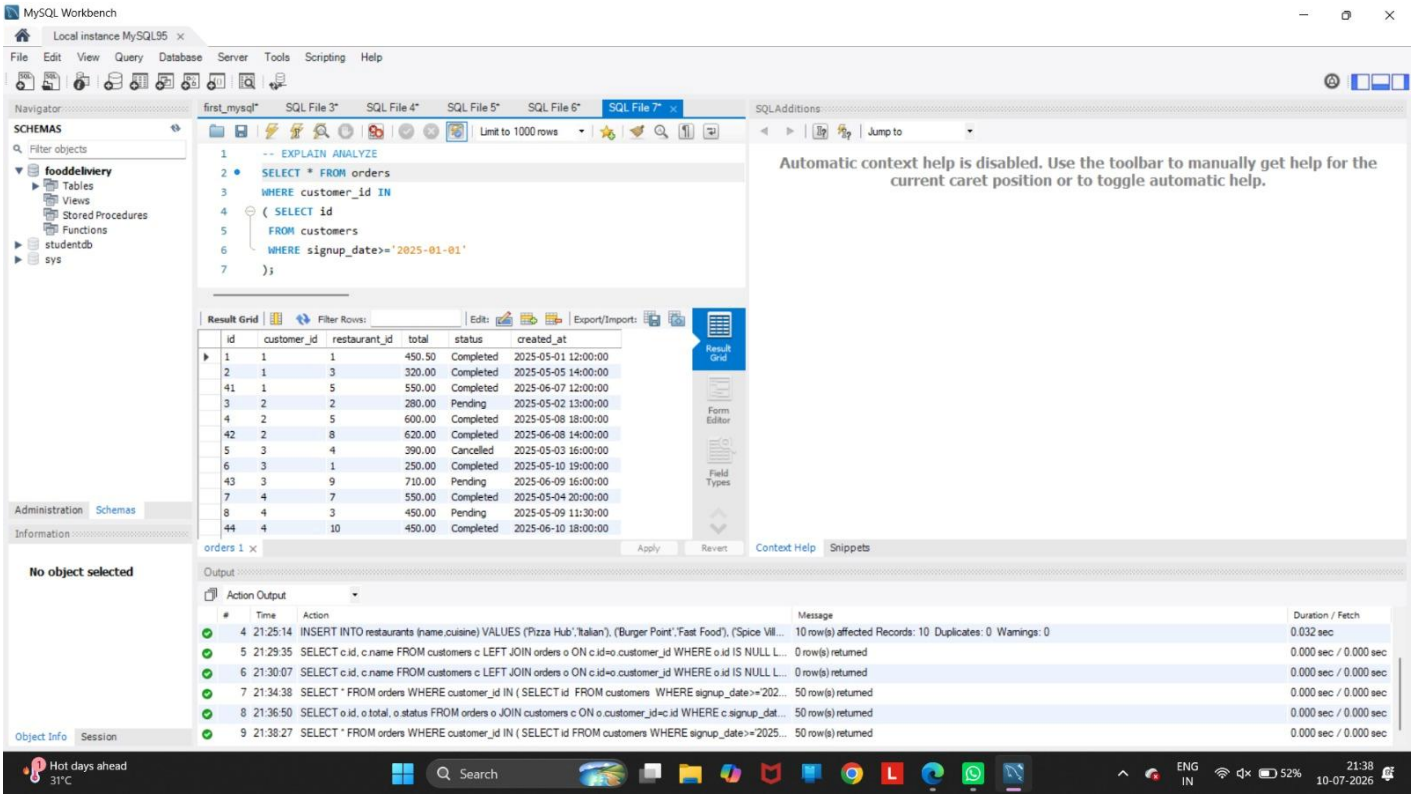


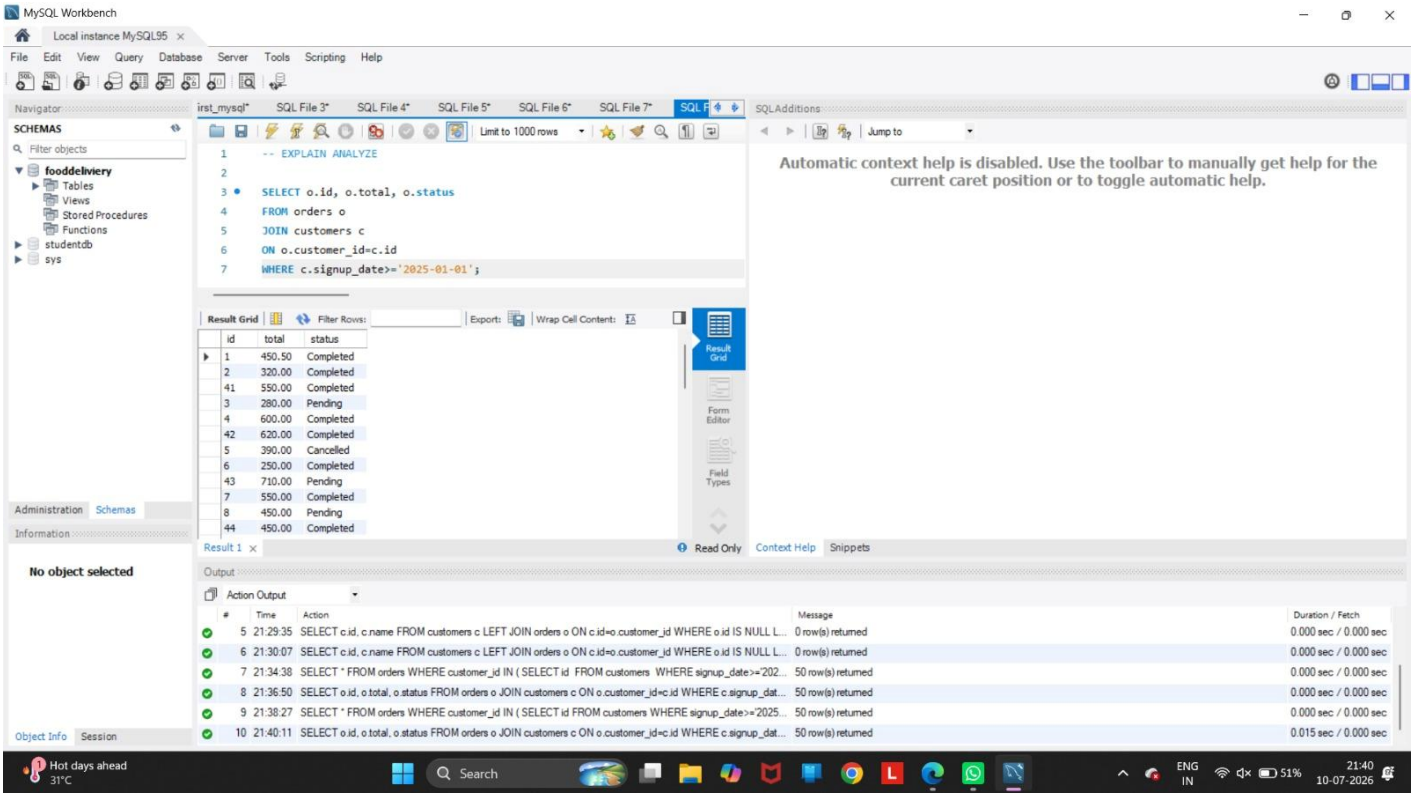


b) Rewrite it again to select only the specific columns you'd actually need on a screen (avoid SELECT *).



c) Run EXPLAIN ANALYZE on your original query and your final rewritten version, and note what changed.

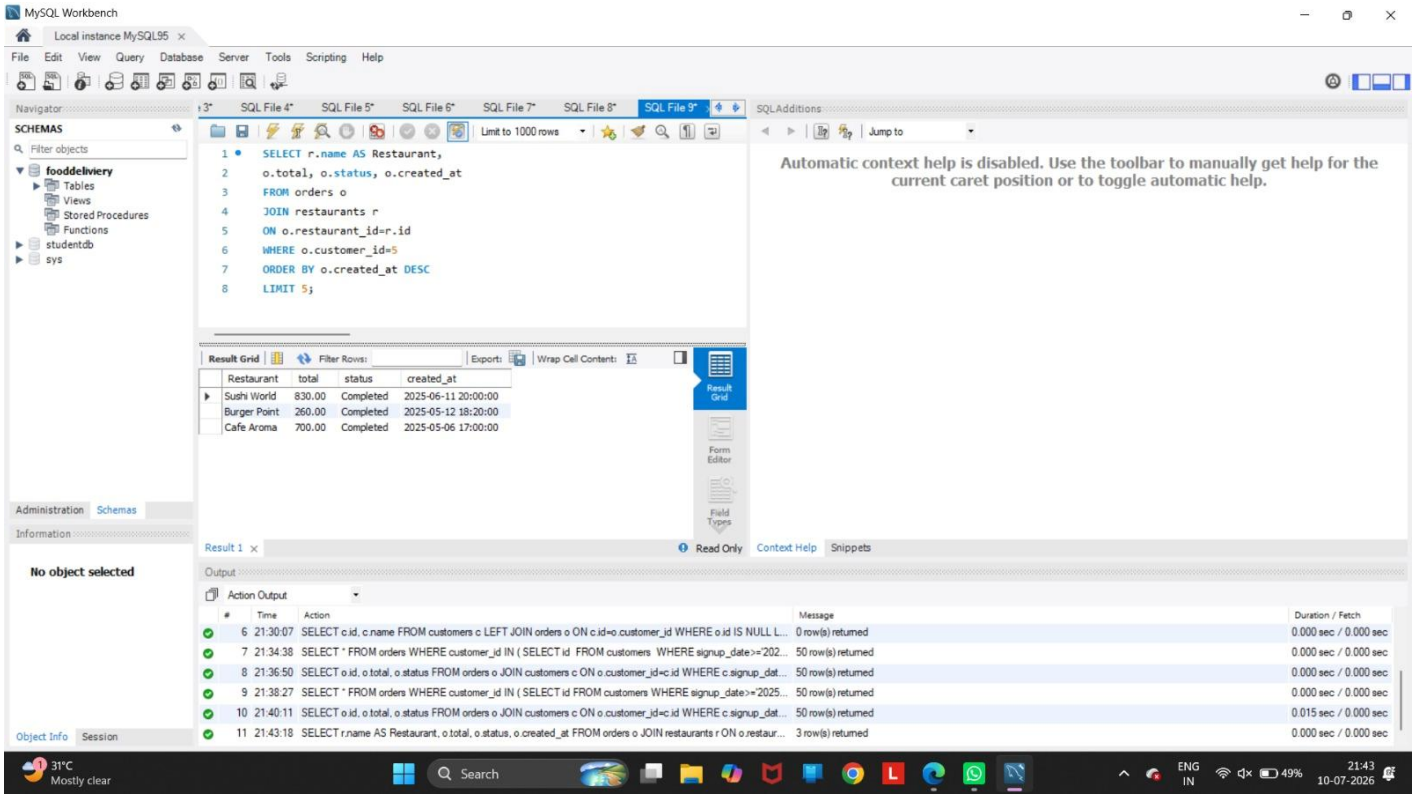

MySQL Workbench
Local instance MySQL95
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
fooddeliviery
Tables
Views
Stored Procedures
Functions
studentdb
sys
Administration Schemas
Information
No object selected
Object Info Session
first_mysql* SQL File 3* SQL File 4* SQL File 5* SQL File 6* SQL File 7*
Limit to 1000 rows
1 -- EXPLAIN ANALYZE
2 SELECT * FROM orders
3 WHERE customer_id IN
4 (SELECT id
5 FROM customers
6 WHERE signup_date>='2025-01-01'
7);
Result Grid Filter Rows: Edit: Export/Import:
id customer_id restaurant_id total status created_at
1 1 1 450.50 Completed 2025-05-01 12:00:00
2 1 3 320.00 Completed 2025-05-05 14:00:00
41 1 5 550.00 Completed 2025-06-07 12:00:00
3 2 2 280.00 Pending 2025-05-02 13:00:00
4 2 5 600.00 Completed 2025-05-08 18:00:00
42 2 8 620.00 Completed 2025-06-08 14:00:00
5 3 4 390.00 Cancelled 2025-05-03 16:00:00
6 3 1 250.00 Completed 2025-05-10 19:00:00
43 3 9 710.00 Pending 2025-06-09 16:00:00
7 4 7 550.00 Completed 2025-05-04 20:00:00
8 4 3 450.00 Pending 2025-05-09 11:30:00
44 4 10 450.00 Completed 2025-06-10 18:00:00
orders 1
Apply Revert
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
4 21:25:14 INSERT INTO restaurants (name,cuisine) VALUES ('Pizza Hub','Italian'), ('Burger Point','Fast Food'), ('Spice Vill... 10 row(s) affected Records: 10 Duplicates: 0 Warnings: 0 0.032 sec
5 21:29:35 SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L... 0 row(s) returned 0.000 sec / 0.000 sec
6 21:30:07 SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L... 0 row(s) returned 0.000 sec / 0.000 sec
7 21:34:38 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>='202... 50 row(s) returned 0.000 sec / 0.000 sec
8 21:36:50 SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat... 50 row(s) returned 0.000 sec / 0.000 sec
9 21:38:27 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>='2025... 50 row(s) returned 0.000 sec / 0.000 sec
Hot days ahead 31°C
Search
ENG IN 52% 21:38 10-07-2026


MySQL Workbench
Local instance MySQL95
File Edit View Query Database Server Tools Scripting Help
Navigator
SCHEMAS
Filter objects
fooddeliviery
Tables
Views
Stored Procedures
Functions
studentdb
sys
Administration Schemas
Information
No object selected
Object Info Session
irst_mysql* SQL File 3* SQL File 4* SQL File 5* SQL File 6* SQL File 7* SQL F
Limit to 1000 rows
1 -- EXPLAIN ANALYZE
2
3 SELECT o.id, o.total, o.status
4 FROM orders o
5 JOIN customers c
6 ON o.customer_id=c.id
7 WHERE c.signup_date>='2025-01-01';
Result Grid Filter Rows: Export: Wrap Cell Content:
id total status
1 450.50 Completed
2 320.00 Completed
41 550.00 Completed
3 280.00 Pending
4 600.00 Completed
42 620.00 Completed
5 390.00 Cancelled
6 250.00 Completed
43 710.00 Pending
7 550.00 Completed
8 450.00 Pending
44 450.00 Completed
Result 1
Read Only
SQLAdditions
Jump to
Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.
Context Help Snippets
Output
Action Output
Time Action Message Duration / Fetch
5 21:29:35 SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L... 0 row(s) returned 0.000 sec / 0.000 sec
6 21:30:07 SELECT c.id, c.name FROM customers c LEFT JOIN orders o ON c.id=o.customer_id WHERE o.id IS NULL L... 0 row(s) returned 0.000 sec / 0.000 sec
7 21:34:38 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>='202... 50 row(s) returned 0.000 sec / 0.000 sec
8 21:36:50 SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat... 50 row(s) returned 0.000 sec / 0.000 sec
9 21:38:27 SELECT * FROM orders WHERE customer_id IN (SELECT id FROM customers WHERE signup_date>='2025... 50 row(s) returned 0.000 sec / 0.000 sec
10 21:40:11 SELECT o.id, o.total, o.status FROM orders o JOIN customers c ON o.customer_id=c.id WHERE c.signup_dat... 50 row(s) returned 0.015 sec / 0.000 sec
Hot days ahead 31°C
Search
ENG IN 51% 21:40 10-07-2026

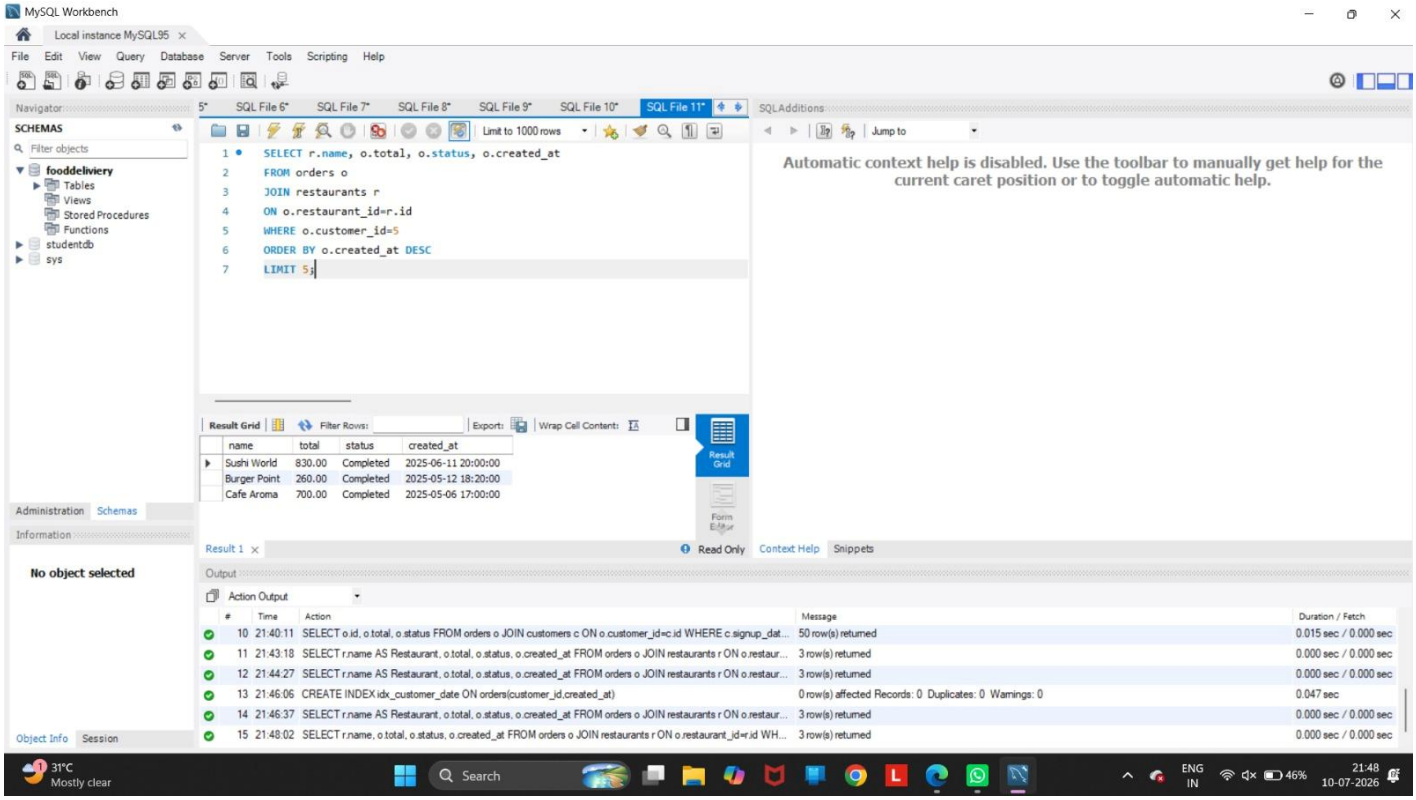
PART 3

TASK 5 The Customer Profile Page

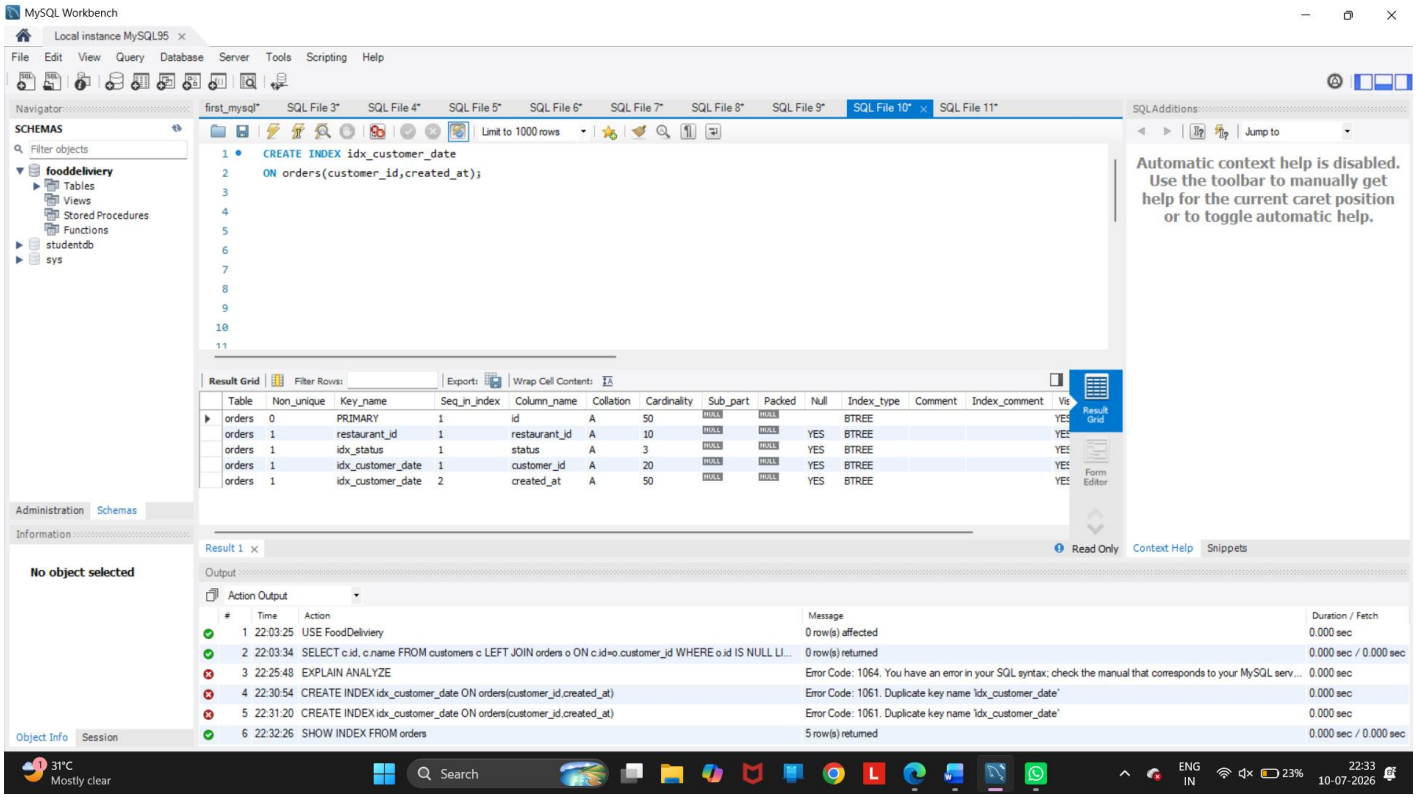
a) Write a query that joins orders with restaurants (and customers, if needed) to return: restaurant name, order total, order status, and created_at — filtered to one specific customer, sorted by the most recent order first, limited to 5 results.



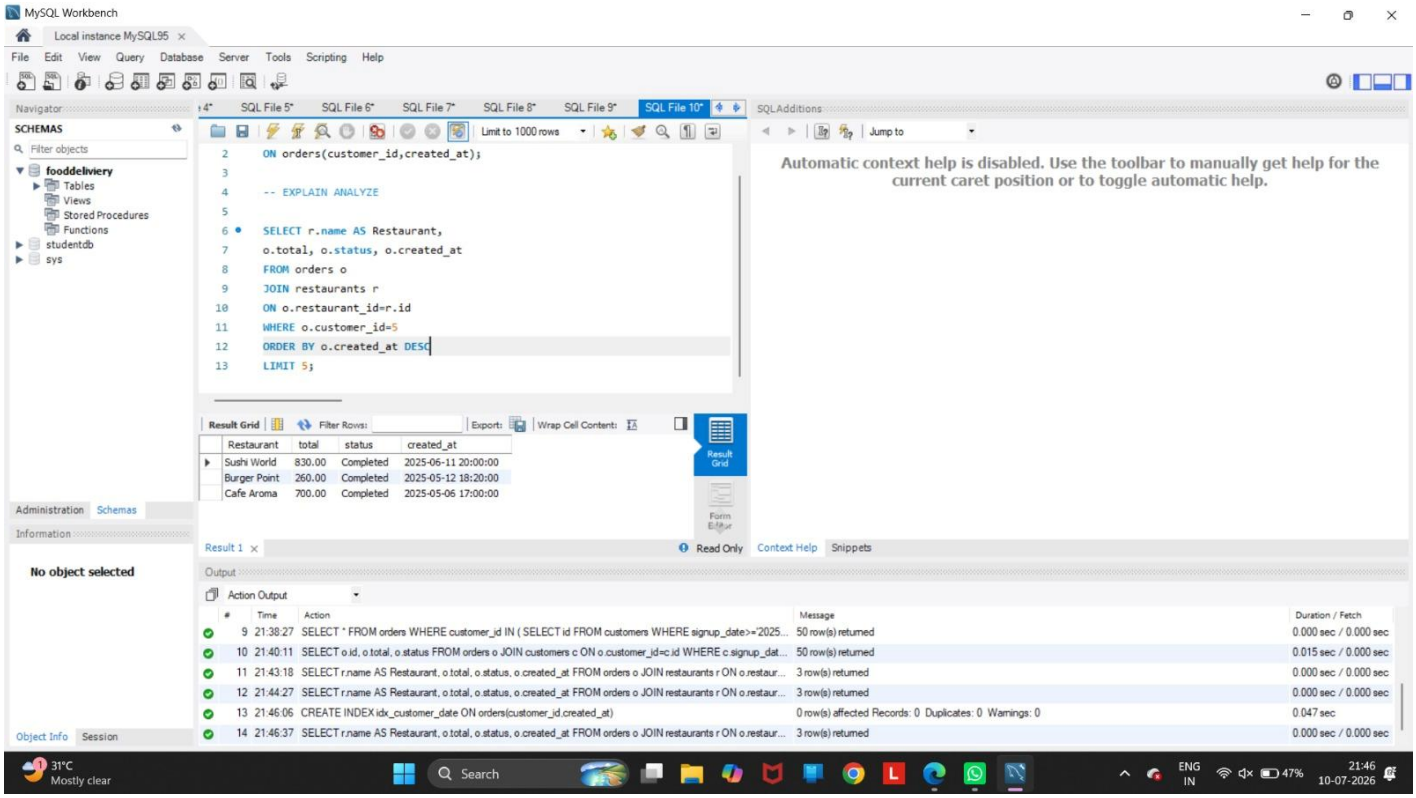
b) Run EXPLAIN ANALYZE on it and record the rows examined and time taken.



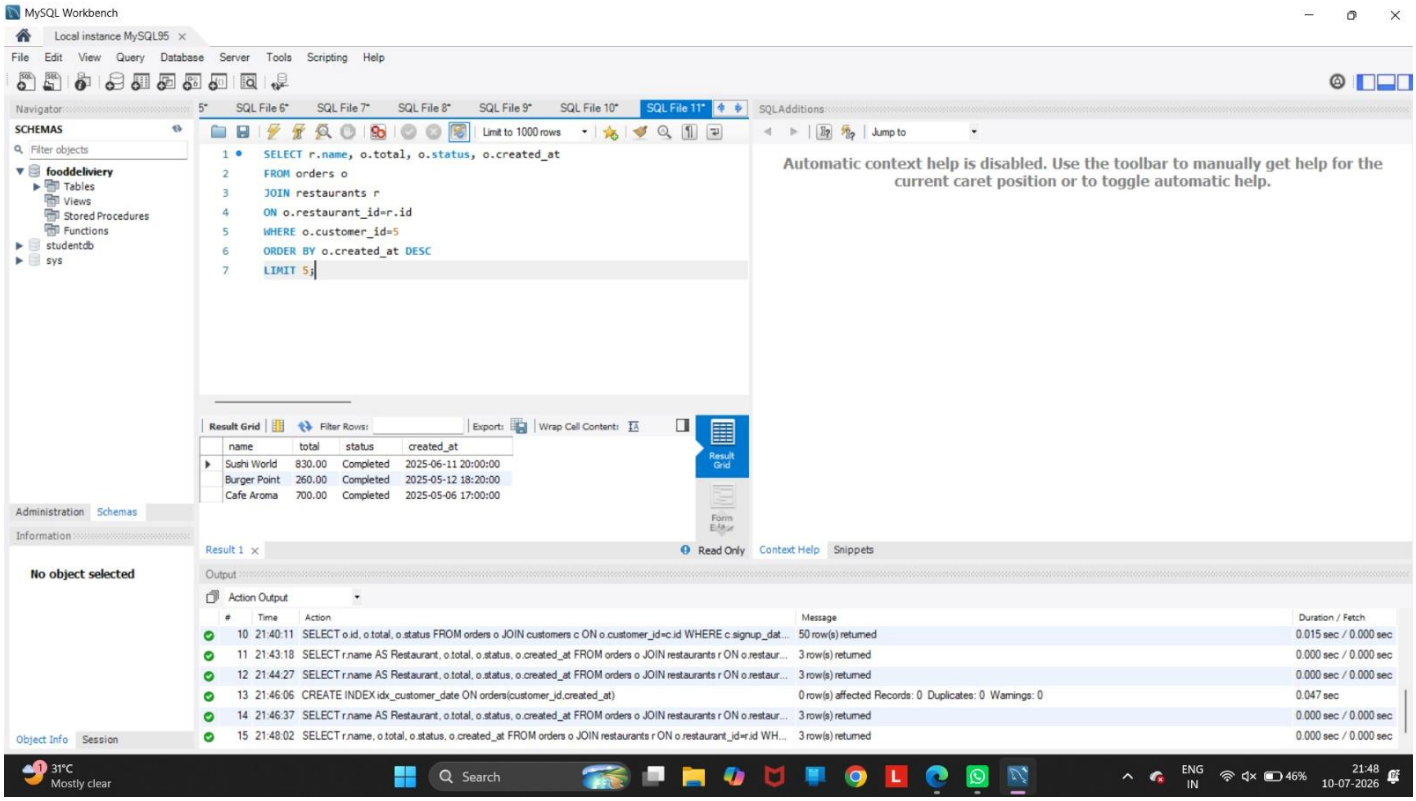
c) Create the index (or indexes) you think will make this query fast. Think about which column(s) appear in your WHERE clause and your ORDER BY.



d) Re-run the same query and compare the new numbers to step (b).



e) Rewrite the query to select only the exact columns the profile page needs — no SELECT *.



f) Submit your before/after comparison, plus 2–3 sentences explaining why the index you chose helped.

Before indexing, MySQL scanned many rows to find the customer's orders and sort them by date. After creating a composite index on (customer_id, created_at), MySQL could quickly locate the matching rows in the required order, reducing both the number of rows examined and the execution time.